

Guia de presentacion por materias

Indice de materias

```
| Indice | Materia | Documento base |  
|---|---|---|  
| MET | Metodologia de la Programacion | ANALISIS_METODOLOGIA_PROGRAMACION.md |  
| FUN | Fundamentos Matematicos | ANALISIS_FUNDAMENTOS.md |  
| ALG | Algebra y Trigonometria | ANALISIS_ALGEBRA_TRIGONOMETRIA.md |  
| CAL | Calculo | ANALISIS_CALCULO.md |  
| FIS | Fisica | ANALISIS_FISICA.md |
```

MET - Metodologia de la Programacion

Idea para explicar:

Metodologia de la programacion incide en como se estructura el proyecto para que funcione como un programa completo. El sistema tiene entradas, procesamiento y salidas. Las entradas no se piden con input, porque el objetivo es simular monitoreo en tiempo real. En lugar de capturar datos manualmente, el programa genera paquetes durante 60 segundos.

El flujo del programa puede explicarse asi:

Entradas simuladas -> procesamiento -> reglas -> salidas

Las entradas simuladas son paquetes, intentos de login, CPU, cambios de archivos, salida de datos, usuario, IP y tipo de incidente. Despues el programa procesa esos datos usando funciones, condicionales, ciclos, listas y diccionarios. Finalmente entrega estados de seguridad, alertas, protecciones, graficas y un CSV.

Procedimiento que se puede mostrar:

1. Generar datos del paquete
2. Evaluar si hay ataque
3. Calcular metricas
4. Clasificar el estado
5. Guardar historial
6. Generar graficas y CSV

Estados como salida del programa:

```
| Estado | Interpretacion para presentacion |
```

```
|---|---|
| NORMAL | seguridad alta |
| SOSPECHOSO | seguridad media |
| ALERTA | seguridad baja |
| CRITICO | seguridad critica |
```

Apoyo visual sugerido:

```
| Material | Para que sirve |
|---|---|
| UML 01_flujo_general.drawio | explicar el orden del programa |
| consola del programa | mostrar entradas procesadas y alertas |
| CSV pdi_slp_reporte_tabla.csv | demostrar que el programa guarda resultados |
| resumen final | enseñar incidentes, IPs bloqueadas y estado final |
```

FUN - Fundamentos Matematicos

Idea para explicar:

Fundamentos matematicos es una de las materias que mas abarca el proyecto, porque aparece en la forma en que el sistema clasifica datos, usa conjuntos, aplica reglas logicas, trabaja con umbrales y justifica el cifrado modular. No solo se trata de decir "hay un if", sino de explicar como los datos se convierten en decisiones.

Primero, el sistema recibe datos del paquete: paquetes por segundo, intentos de login, usuario, IP, salida de datos, cambios de archivos, temperatura y flujo. Esos datos se comparan contra umbrales. Con eso se decide si el estado es NORMAL, SOSPECHOSO, ALERTA o CRITICO.

La parte logica se puede presentar con proposiciones:

```
p = trafico elevado
q = intentos de login elevados
r = flujo digital critico
s = temperatura critica
u = usuario autorizado
```

Con esas proposiciones, las reglas pueden explicarse de esta manera:

```
(p AND q) -> ALERTA
(p OR r) -> CRITICO
s -> REFRIGERACION ACTIVADA
NOT u -> ALERTA
```

Tambien entra el tema de conjuntos. El sistema trabaja con usuarios autorizados, usuarios sospechosos, IPs internas, IPs sospechosas e IPs bloqueadas. Cuando llega un paquete, se revisa pertenencia:

usuario pertenece a agentes_autorizados -> acceso valido
usuario no pertenece a agentes_autorizados -> acceso no autorizado
IP sospechosa detectada -> puede agregarse a IPs bloqueadas

Esto permite explicar clasificacion y relaciones entre conjuntos. El paquete no se analiza como dato aislado, sino como elemento que puede pertenecer o no a ciertos grupos.

Otro punto importante en fundamentos es el cifrado. El programa usa modularidad para proteger el nombre del registro cuando el estado sube de riesgo. En estado normal no cifra; en alerta o sospechoso usa cifrado Cesar; en critico usa cifrado reforzado con modulo.

Formula de apoyo para cifrado multiplicativo:

$$C = aP \bmod 26$$

Formula de apoyo para descifrado:

$$P = C * a^{-1} \bmod 26$$

Esto se puede explicar como una aplicacion de congruencias modulares. Las letras siempre quedan dentro del alfabeto porque se trabaja modulo 26. Los numeros se mueven con modulo 10 para que sigan siendo cifras del 0 al 9. El inverso modular permite que el servidor autorizado recupere el nombre original del archivo.

Procedimiento matematico general:

1. Recibir o generar datos del paquete
2. Revisar pertenencia a conjuntos
3. Comparar datos contra umbrales
4. Aplicar reglas logicas
5. Clasificar el estado del sistema
6. Activar protecciones
7. Cifrar o descifrar registros segun el estado
8. Verificar resultados con graficas y CSV

Ejemplo para decir:

Si un paquete tiene trafico alto y tambien muchos intentos de login, no se toma como evento aislado. La combinacion de ambas condiciones activa una alerta. Si ademas el flujo digital rebasa el nivel critico, el sistema cambia a CRITICO y

activa cifrado reforzado. En ese momento fundamentos no solo aparece en la logica, tambien aparece en la modularidad del cifrado y en la clasificacion por conjuntos.

Tambien se puede mencionar que las graficas sirven para comprobar si las reglas tienen coherencia. Por ejemplo, si la grafica de paquetes sube demasiado y al mismo tiempo la grafica de estados cambia a alerta o critico, entonces la decision del sistema se puede justificar visualmente.

Apoyo visual sugerido:

```
| Material | Para que sirve |
|---|---|
| grafica 04_estado_sistema_vs_tiempo.png | mostrar como cambian las categorias logicas |
| grafica 01_paquetes_vs_tiempo.png | justificar la proposicion p = trafico elevado |
| grafica 02_intentos_login_vs_tiempo.png | justificar la proposicion q = login elevado |
| grafica 05_indice_flujo_digital_vs_tiempo.png | mostrar como un indice ayuda a decidir riesgo |
| UML 04_estados_protecciones.drawio | explicar decision, alerta y proteccion |
| CSV con columnas estado, cifrado y direccion_servidor | mostrar que la clasificacion y el descifrado queda
n guardados |
| formula (p AND q) -> ALERTA | explicar logica proposicional |
| formula C = aP mod 26 | explicar cifrado modular |
| formula P = C * a^-1 mod 26 | explicar recuperacion con inverso modular |
```

ALG - Algebra y Trigonometria

Idea para explicar:

Algebra y trigonometria incide en la forma de modelar datos. En algebra, el programa combina varias variables para generar un indice de flujo digital. Ese indice resume la actividad del sistema en un solo numero.

La formula que se puede mostrar es:

$$\text{Flujo digital} = 0.45P + 0.25L + 0.15A + 0.15D$$

Donde:

P = paquetes por segundo
L = intentos de login
A = cambios en archivos
D = salida de datos

Esta formula permite explicar que no todos los datos pesan igual. Los paquetes tienen mayor peso porque representan la carga principal de la red. Los login, archivos y salida de datos complementan el analisis.

En trigonometria se usa una senal periodica como referencia:

$$S(t) = A \sin(\omega t) + B \cos(\omega t)$$

Esa senal sirve para comparar el comportamiento real contra un patron oscilatorio. Si el trafico se separa demasiado, se interpreta como desviacion.

Procedimiento matematico:

1. Tomar datos del paquete
2. Sustituirlos en la formula del flujo digital
3. Comparar el flujo contra umbrales
4. Generar una senal periodica
5. Comparar paquetes reales contra el patron

Apoyo visual sugerido:

```
| Material | Para que sirve |  
|---|---|  
| formula del flujo digital | mostrar el modelo algebraico |  
| grafica 05_indice_flujo_digital_vs_tiempo.png | enseñar el resultado del modelo algebraico |  
| formula  $S(t) = A \sin(\omega t) + B \cos(\omega t)$  | mostrar el modelo trigonometrico |  
| grafica 07_analisis_oscilatorio_vs_tiempo.png | comparar patron regular contra comportamiento real |
```

CAL - Calculo

Idea para explicar:

Calculo incide en el analisis de cambio. El sistema no solo revisa cuantos paquetes hay, sino cuanto cambian de un segundo al siguiente. Eso permite detectar subidas bruscas.

La funcion principal se interpreta como:

$$T(t) = \text{paquetes en el segundo } t$$

La tasa de cambio se calcula asi:

$$\Delta T = T(t) - T(t - 1)$$

Como el sistema avanza de segundo en segundo, la derivada discreta se representa como:

$$T'(t) \text{ aproximada} = (T(t) - T(t - 1)) / 1$$

Tambien se usa segunda derivada:

$$T''(t) \text{ aproximada} = T'(t) - T'(t - 1)$$

La primera derivada muestra si el trafico sube o baja. La segunda derivada muestra si ese cambio se acelera o se frena. Ademas se usa la funcion sigmoide para suavizar los cambios:

$$\text{sigmoide}(x) = 1 / (1 + e^{(-x)})$$

Procedimiento matematico:

1. Tomar paquetes actuales
2. Compararlos contra paquetes anteriores
3. Calcular primera derivada
4. Calcular segunda derivada
5. Suavizar con sigmoide
6. Comparar contra umbral de alerta

Apoyo visual sugerido:

Material	Para que sirve
grafica 06_derivadas_trafico_vs_tiempo.png	mostrar primera y segunda derivada
formula Delta T = T(t) - T(t-1)	explicar tasa de cambio
formula de sigmoide	explicar suavizado
grafica 01_paquetes_vs_tiempo.png	comparar datos originales contra cambios

FIS - Fisica

Idea para explicar:

Fisica incide en el impacto material del sistema digital. Aunque el proyecto es de ciberseguridad, un ataque tambien puede aumentar la carga del servidor. Si suben los paquetes, intentos de login o salida de datos, sube el CPU y despues la temperatura.

El primer modelo fisico es:

Temperatura = Temperatura ambiente + k(CPU)

En el proyecto:

Temperatura ambiente = 28 C

k = 0.58

Tambien se usa enfriamiento pasivo:

Temperatura nueva = Temperatura actual - 1 C cada 5 segundos

Para el enfriamiento activo se usa continuidad:

$A_1 v_1 = A_2 v_2$

$v_2 = (A_1 v_1) / A_2$

Y Bernoulli:

$P_1 + 1/2 \rho v_1^2 = P_2 + 1/2 \rho v_2^2$

El programa usa esa relacion para estimar velocidad y presion del refrigerante.

Despues calcula un descenso de temperatura a partir de la caida relativa de presion:

$\text{Descenso} = \text{coeficiente} * \text{velocidad} * ((P_1 - P_2) / P_1)$

Procedimiento fisico:

1. Actividad digital aumenta CPU
2. CPU aumenta temperatura
3. Temperatura se compara con umbrales
4. Si hay riesgo, se activa refrigeracion
5. La refrigeracion reduce la temperatura

Apoyo visual sugerido:

Material	Para que sirve
---	---

grafica 03_temperatura_vs_tiempo.png	mostrar temperatura con y sin enfriamiento
formula de temperatura	explicar relacion CPU-temperatura
formula de continuidad	justificar velocidad del refrigerante
formula de Bernoulli	justificar presion del refrigerante
UML 05_refrigeracion.drawio	explicar el flujo de enfriamiento